Laboratory Activity # 1: From Use Case to Working IT System

Laboratory Title: Design and Develop a Simple IT Help Desk Ticketing System
Suggested Duration: 2–3 hours
Programming Language: Any available language — Python, Java, C#, JavaScript, PHP, C++, etc.

I. Laboratory Scenario

Your university's IT Department receives concerns from students and faculty through chat, email, and personal visits. The department wants a simple **IT Help Desk Ticketing System** to organize these requests.

The system should allow a user to submit an IT concern and allow IT personnel to view, update, and resolve submitted tickets.

Examples of concerns include:

"I cannot connect to the Wi-Fi."
"I forgot my LMS password."
"The computer in Laboratory 2 is not working."
"I cannot access my student portal."

Your team has been assigned to analyze the requirements, model the process, develop a simple prototype, and test it.

II. Learning Objectives

At the end of the laboratory activity, students should be able to:

1. Identify **actors, functional requirements, and use cases** from an IT problem.
2. Construct a simple **Use Case Diagram** and **process model/flowchart**.
3. Translate system requirements into a working program.
4. Demonstrate basic interaction between different components of an information system.
5. Develop and execute **test cases** based on system requirements.

III. System Requirements

1. Students must create a prototype with two actors:

User

- Create a support ticket
- View submitted tickets

IT Support

- View all tickets
- Update ticket status

- Mark a ticket as resolved

2. Each ticket should contain at least: Ticket ID | User Name | Issue Category | Description | Priority | Status

Example:

Ticket ID: T001

User: Juan Dela Cruz

Category: Network

Description: Cannot connect to campus Wi-Fi

Priority: High

Status: Open

For this laboratory, a database is **optional**. Students may store records using arrays, lists, dictionaries, text/CSV files, JSON files, or a database depending on their programming level.

IV. Part A — Requirements Analysis

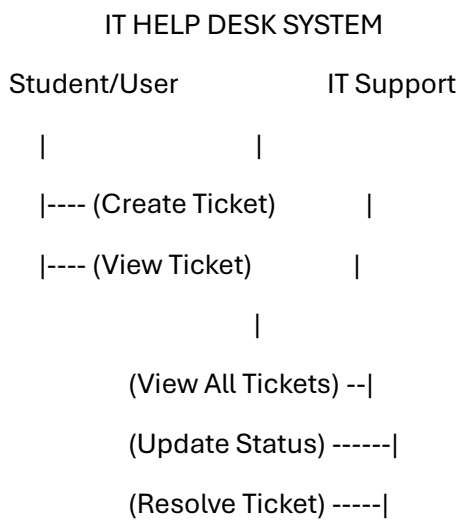
1. Before writing code, students must identify the requirements. Give students this question: **"What should the system do before we decide how to program it?"** They should produce a table like:

ID	Functional Requirement
FR-01	System shall allow users to create a support ticket.
FR-02	System shall automatically generate a Ticket ID.
FR-03	System shall allow users to select an issue category.
FR-04	System shall allow users to assign a priority.
FR-05	System shall allow IT Support to view tickets.
FR-06	System shall allow IT Support to update ticket status.
FR-07	System shall display resolved tickets.

For example, students might suggest search, timestamps, authentication, technician assignment, or notifications.

V. Part B — Create the Use Case Model

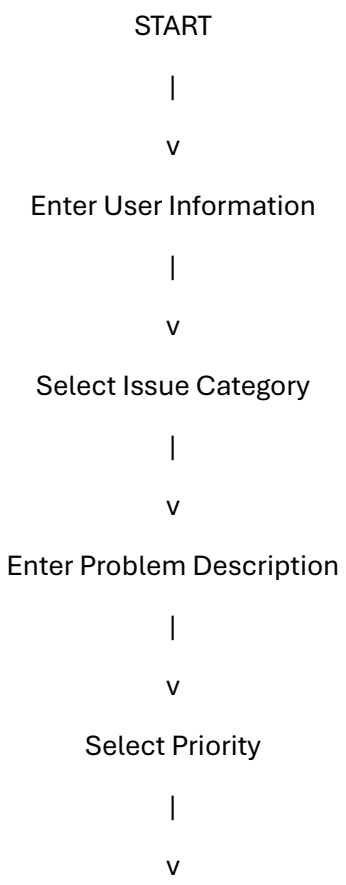
Students create this basic model:

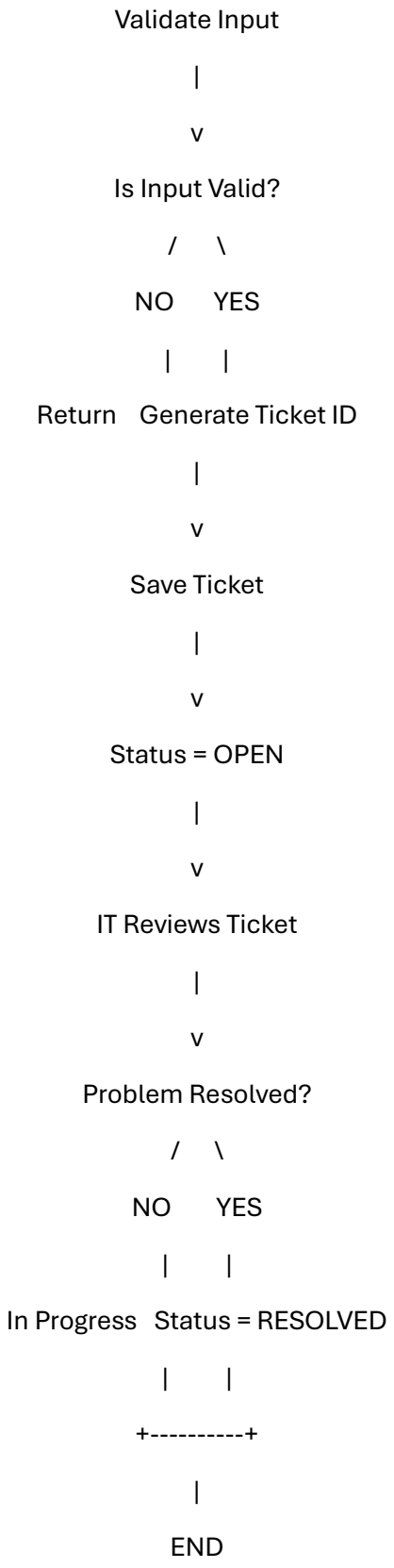


They may use any available modeling tool or draw the diagram manually.

VI. Part C — Create a Process Flowchart

Students should model the ticket workflow:




Validate Input
Is Input Valid?
NO
YES
Return
Generate Ticket ID
Save Ticket
Status = OPEN
IT Reviews Ticket
Problem Resolved?
NO
YES
In Progress
Status = RESOLVED
END

VII. Part D — Programming Challenge

Students now translate the model into code.

Minimum Program Menu

==========

IT HELP DESK SYSTEM

==========

[1] Create Ticket

[2] View Tickets

[3] Update Ticket Status

[4] Search Ticket

[5] Exit

Select Option:

Example Interaction

Select Option: 1

--- CREATE SUPPORT TICKET ---

Name: Maria Santos

Issue Category:

[1] Network

[2] Hardware

[3] Software

[4] Account

[5] Others

Select: 1

Describe the problem:

Cannot connect to campus Wi-Fi.

Priority:

[1] Low

[2] Medium

[3] High

Select: 3

Ticket successfully created!

Ticket ID: T001

Status: OPEN

VIII. Sample Python Implementation

You can give this as a **starter program** rather than the completed solution so students still have to develop parts themselves.

tickets = []ticket_counter = 1def create_ticket(): global ticket_counter print("\n--- CREATE IT SUPPORT TICKET ---") name = input("User Name: ") category = input("Category (Network/Hardware/Software/Account): ") description = input("Describe the problem: ") priority = input("Priority (Low/Medium/High): ") ticket = { "id": f"T{ticket_counter:03}", "user": name, "category": category, "description": description, "priority": priority, "status": "OPEN" } tickets.append(ticket) print("\nTicket successfully created!") print("Ticket ID:", ticket["id"]) ticket_counter += 1def view_tickets(): print("\n--- SUPPORT TICKETS ---")

Students using **Java, C#, C++, JavaScript, PHP, or another language** can implement the same requirements.

IX. Required Student Deliverables

Each group submits **one laboratory report** containing:

1. Problem description and functional requirements
2. Use Case Diagram
3. Process Flowchart

4. Source code
5. Screenshots of the running program
6. At least five test cases with results
7. Explanation of the integration component
8. Short reflection answering:

“How did the requirements and system models help you develop and test the application?”

Suggested Grading Rubric — 100 Points

Criterion	**Points**
Requirements Analysis	15
Use Case Model	15
Process/Flowchart Model	15
Working Program	25
System Integration Feature	15
Testing and Test Cases	10
Documentation / Presentation	5
TOTAL	**100**

For a stronger **Systems Integration and Architecture 1** activity, I would make the **integration feature mandatory** rather than grading only the source code. That connects the programming exercise directly to the course instead of turning it into a basic programming laboratory.